

CS673 Software Engineering
Team 2 - HealthNest
Software Design Document

<u>Team Member</u>	<u>Role(s)</u>	<u>Signature</u>	<u>Date</u>
Deven Shah	Team Lead, Requirements Lead	<i>Deven Shah</i>	5/17/2026
Stephanie Wang	QA Lead, Design & Implementation Lead	<i>Stephanie Wang</i>	5/17/2026
Aaron Brown	Security Lead, Design & Implementation Lead	<i>Aaron Brown</i>	5/16/2026
Chamarr Auber	Team Lead, Configuration Lead	<i>Chamarr Auber</i>	5/17/2026

Revision history

<u>Version</u>	<u>Author</u>	<u>Date</u>	<u>Change</u>
1.0	Chamarr Auber, Aaron Brown, Deven Shah, Stephanie Wang	5/26/2026	Initialized all sections of this document.
<u>2.0</u>		<u>6/16/2026</u>	

- [Introduction](#)
- [Software Architecture](#)
- [Class Diagram](#)
- [UI Design \(if applicable\)](#)
- [Database Design \(if applicable\)](#)
- [Security Design](#)
- [Business Logic and/or Key Algorithms](#)
- [Design Patterns](#)
- [Rest APIs](#)
- [Any Additional Topics you would like to include.](#)
- [AI usage Log](#)
- [References](#)
- [Glossary](#)

1. Introduction

This document explains the software design for HealthNest, an AI-powered healthcare platform. HealthNest is designed as one connected medical dashboard for both patients and providers. The main problem we are trying to address is that healthcare information is often spread across different systems, which can make it harder for patients to manage their care and harder for providers to keep track of their daily work.

The goal of HealthNest is to bring the most important healthcare tasks into one connected platform, especially appointments, lab results, medications, secure messaging, billing, and AI-assisted support. Patients can use the system to keep track of appointments, understand lab result instructions, review medications, and get AI-supported guidance for next steps. Providers can use their dashboard to review patient records, manage lab result follow-up, respond to messages, and organize daily clinical tasks. Each role has its own dashboard so the information is shown based on what that user needs to do, instead of making them move through several disconnected pages or systems.

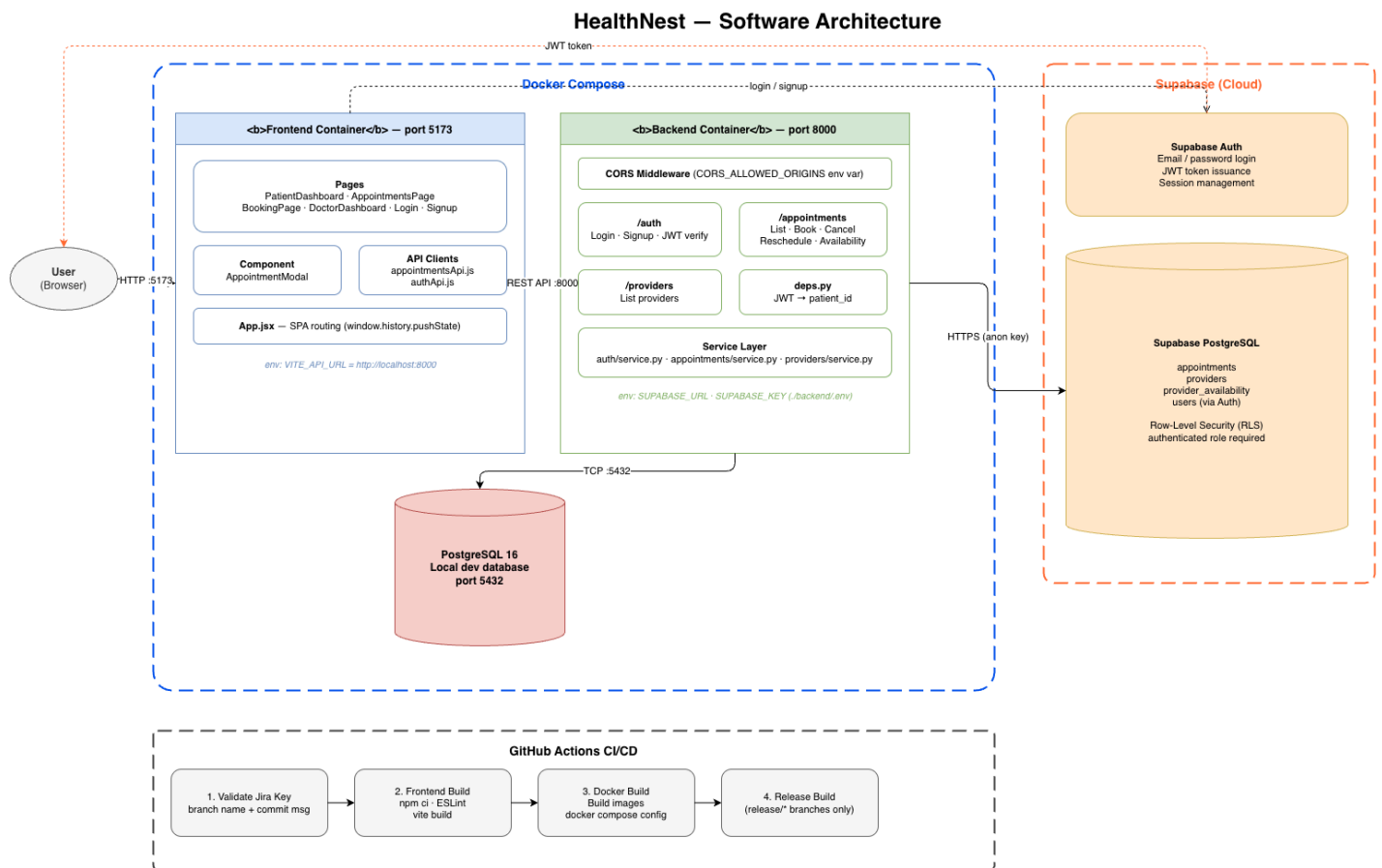
This document covers the main design areas for the HealthNest system, including:

- Software architecture and major system components
- Class diagrams for the main parts of the platform
- UI design, including the visual style and Figma mockups
- Database design based on the Supabase schema
- Security design, including authentication, authorization, and data protection
- Key business logic shown through flowcharts
- Design patterns used in the frontend and backend

2. Software Architecture

HealthNest is designed using a modern containerized client-server architecture that separates the frontend presentation layer, backend business logic layer, authentication services, and database infrastructure. The application is deployed and managed using Docker Compose for local development and integrates with Supabase cloud services for authentication and persistent data storage.

The system consists of three primary layers: the frontend client application, the backend REST API service, and the database/authentication infrastructure. These layers are shown in the diagram below.



Frontend Architecture

The frontend is hosted within a dedicated Docker container running on port 5173 and is built as a Single Page Application (SPA). The frontend is responsible for rendering the user interface,

handling client-side routing, and communicating with backend API services.

The frontend contains several application pages including:

- Patient Dashboard
- Appointments Page
- Booking Page
- Doctor Dashboard
- Login Page
- Signup Page

Reusable UI components, such as appointment modal interfaces, are used to improve maintainability and modularity. API communication is abstracted into dedicated client modules responsible for authentication and appointment-related requests.

Client-side routing is managed within the application using browser history navigation to provide seamless SPA behavior without full page reloads.

The frontend communicates with the backend through REST API requests using a configurable environment variable for the API base URL.

Backend Architecture

The backend is hosted within a separate Docker container running on port 8000 and contains the application's core business logic, authentication validation, and API endpoints.

The backend exposes several REST API routes including:

- `/auth` for login, signup, and JWT verification
- `/appointments` for appointment booking, cancellation, rescheduling, and availability management
- `/providers` for retrieving healthcare provider information

A middleware layer handles Cross-Origin Resource Sharing (CORS) configuration to ensure only approved frontend origins can access backend resources.

Authentication dependencies are centralized within shared backend modules that validate JWT tokens and extract authenticated patient information from incoming requests.

The backend follows a layered service architecture in which route handlers delegate business logic to dedicated service modules. This separation improves scalability, maintainability, and testability.

Database and Authentication Infrastructure

The application uses both a local PostgreSQL development database and Supabase cloud services.

For local development, PostgreSQL 16 runs on port 5432 and provides an isolated database environment for testing and development workflows.

Supabase provides:

- Cloud-hosted PostgreSQL services
- User authentication
- JWT token issuance
- Session management

Supabase Authentication handles email/password login workflows and issues JWT access tokens used for authenticated API communication.

The Supabase PostgreSQL database stores:

- User accounts
- Appointments
- Provider information
- Availability schedules

Row-Level Security (RLS) policies are enabled to ensure users can only access authorized healthcare data associated with their accounts.

Authentication Flow

The authentication workflow operates as follows:

1. A user submits login credentials through the frontend application.
2. The frontend sends the authentication request to Supabase Auth.
3. Supabase validates the credentials and returns a JWT token.
4. The frontend stores the JWT token and includes it in subsequent API requests.
5. The backend validates the JWT token before processing protected requests.
6. Authorized users gain access to secured healthcare resources and appointment data.

Containerization and Development Environment

Docker Compose is used to orchestrate the application's local development environment. The Docker Compose configuration manages:

- Frontend container

- Backend container
- PostgreSQL database container

This containerized architecture ensures consistent environments across development machines and simplifies onboarding and deployment workflows.

Continuous Integration and Continuous Deployment (CI/CD)

GitHub Actions is used to automate the application's CI/CD pipeline.

The pipeline includes the following stages:

1. Validation of Jira issue keys in branch names and commit messages
2. Frontend dependency installation, linting, and production build validation
3. Docker image creation and Docker Compose configuration validation
4. Release build generation for release branches

This automated pipeline helps ensure code quality, build consistency, and deployment readiness throughout the development lifecycle.

Architectural Characteristics

The HealthNest architecture emphasizes:

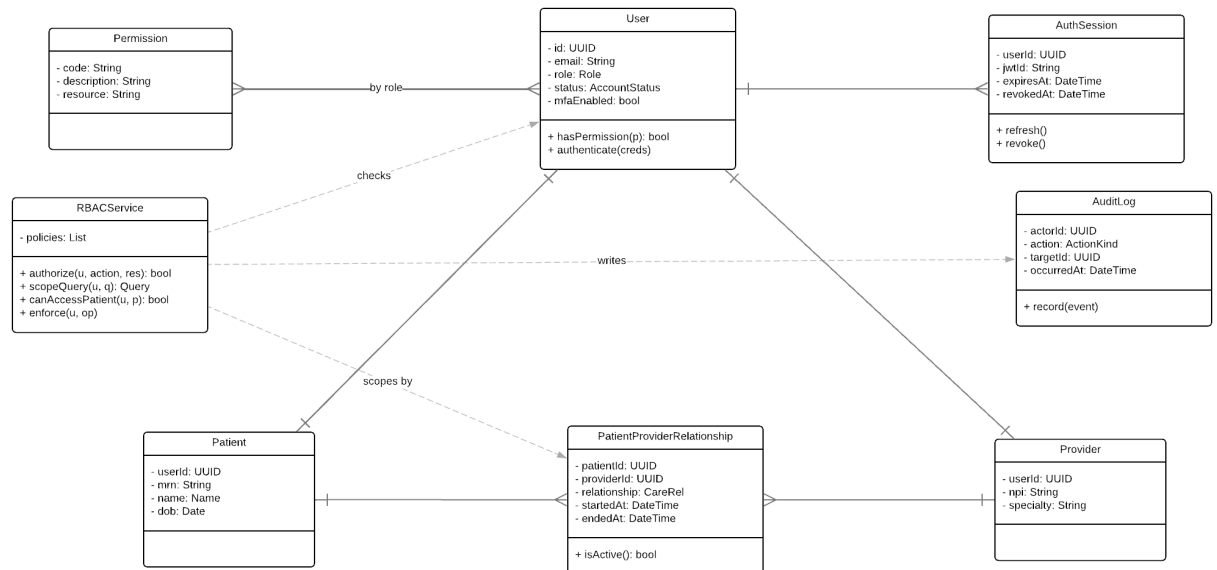
- Modular separation of concerns
- Secure JWT-based authentication
- Scalable REST API design
- Containerized development workflows
- Automated CI/CD validation
- Secure healthcare data access through Row-Level Security policies

This architecture provides a secure, maintainable, and scalable foundation for healthcare appointment management and patient-provider interactions.

3. Class Diagram

Component: Identity-Access Management (IAM)

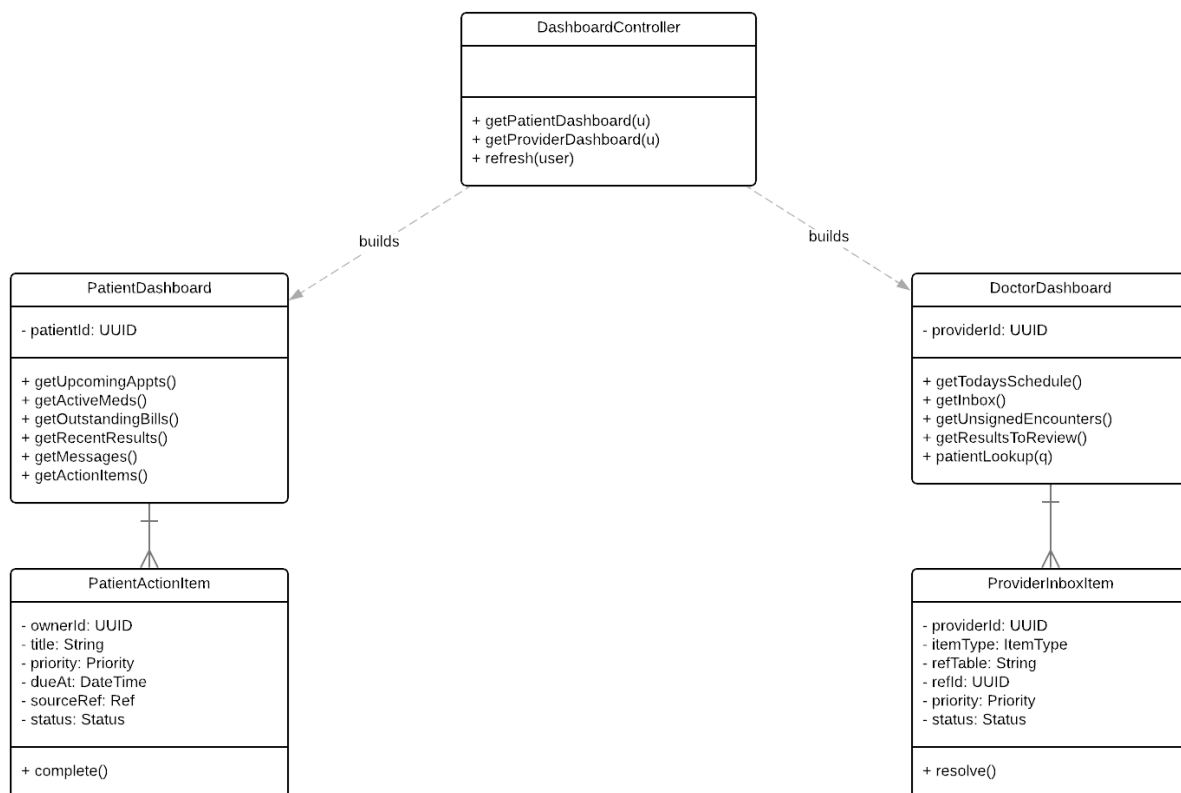
The IAM component is the security spine of the platform. It owns the User aggregate, authentication sessions, and the role-based access control (RBAC) that every other component delegates to and before reading/mutating PHI.



Class	Description
User	Single account for every human (patient or staff). Holds role, status, and MFA configuration. Specialized further by Patient or Provider profiles.
Patient	Profile data and MRN for users who receive care.
Provider	Profile data and NPI for licensed clinicians.
RBACService	Single chokepoint for all authorization decisions.
PatientProviderRelationship	Active care-team edge between a provider and a patient. RBACService scope patient data access by the existence of an active row here.
AuthSession	JWT/refresh-token tracking for rotation, revocation, and logout.
AuditLog	Reference to the cross-cutting audit record written on every authorization decision.

Component: Dashboard

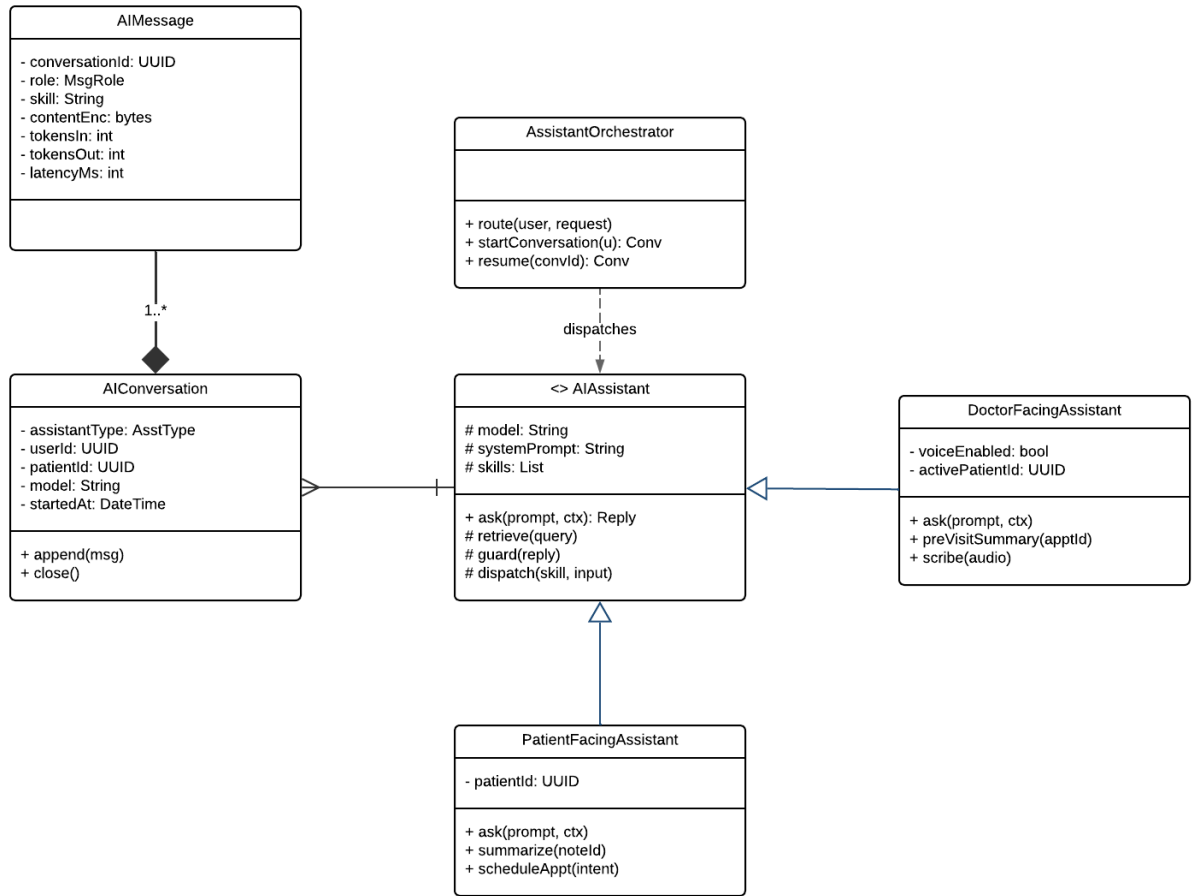
The Dashboard component is the presentation-orchestration layer that serves as the central view component for both patients and doctors. It composes per-role dashboards out of different projections rather than continuously joining tables on every load.



Class	Description
DashboardController	Assembles per-role dashboard views on request.
PatientDashboard	Patient home view: upcoming appointments, active meds, outstanding bills, recent results, messages, prioritized action items.
DoctorDashboard	Provider home view: today's schedule, inbox, unsigned encounters, results to review, patient search.
PatientActionItem	Projection that powers the patient's prioritized to-do list.
ProviderInboxItem	Projection unifying message and encounter entries in one ranked list.

Component: AI Assistant

The AI Assistant component is the core that both the patient-facing and doctor-facing AI assistants extend. AIAssistant is an abstract class that defines the ask/retrieve/guard/dispatch pipeline. PatientFacingAssistant (PFA) and DoctorFacingAssistant (DFA) specialize it with role-appropriate operations. Every interaction is recorded in an AIConversation composed of AIMessage turns.

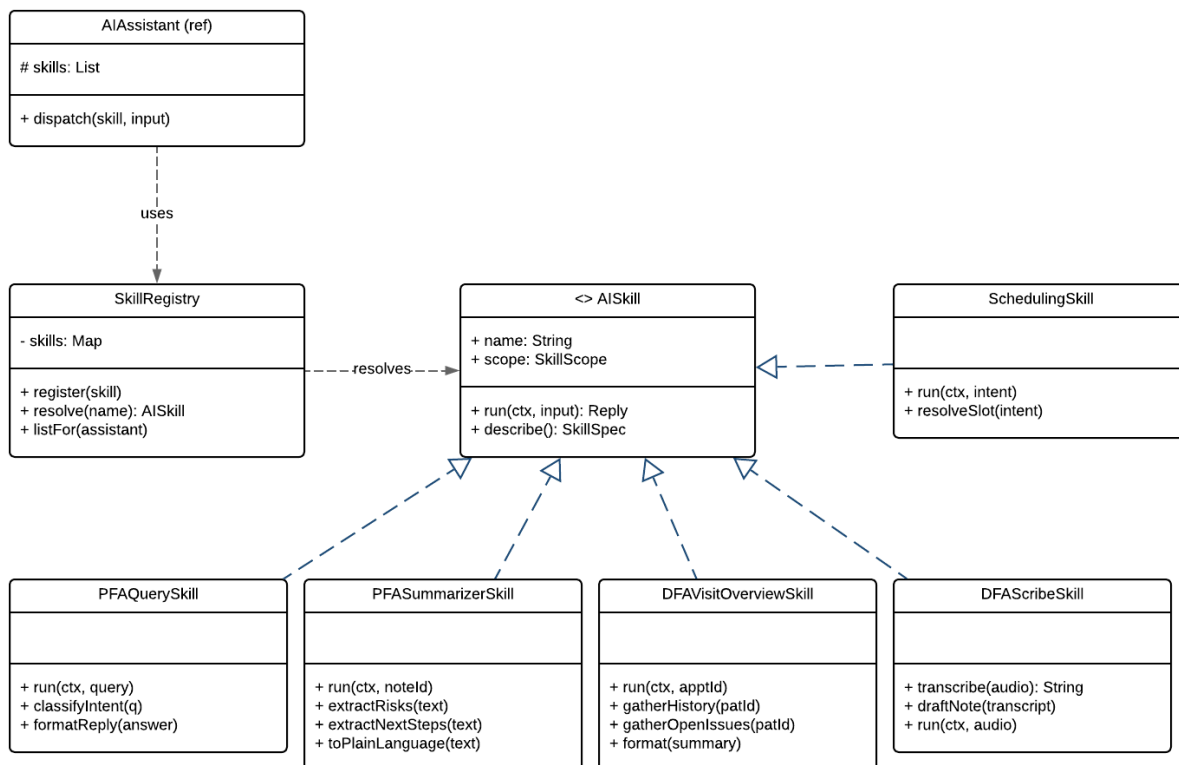


Class	Description
AssistantOrchestrator	Top-level entry point. Resolves the right assistant for the user and starts or resumes the conversation.
AIAssistant	Holds the model, system prompt, and registered skill set.
PatientFacingAssistant	Specialization scoped to a single patient's portfolio. Owns user-facing operations: ask, summarize, scheduleAppt.
DoctorFacingAssistant	Specialization for clinicians. Adds voice mode and the pre-visit summary/scribe operations.

Class	Description
AIConversation	Per-session container. Records assistant type, user, patient context, and model.
AIMessage	Per-turn record. Stores role, content (encrypted), skill used, latency, and token counts.

Component: AI-Assistant Skills

Behaviors that an AI assistant can perform are modeled as Skills. SkillRegistry resolves skill names to implementations, and assistants dispatch to skills rather than embedding behavior in their own methods. This keeps the assistant classes small and lets us add or remove capabilities without touching the assistants.



Class	Description
SkillRegistry	Resolves skill names to implementations and returns the skill set available to a given assistant.
AISkill	Runs the common pipeline using the implementation from the SkillRegistry.

Class	Description
PFAQuerySkill	Conversational question-answering for patients. Classifies intent, retrieves, formats.
PFASummarizerSkill	Plain-language summary of clinician directives, prescriptions, and post-visit notes.
DFAVisitOverviewSkill	Pre-visit patient summary on demand.
DFAscribeSkill	Voice-to-draft-note scribe for clinicians.
SchedulingSkill	Scheduling intents the PFA/DFA can resolve into AppointmentScheduler calls.

4. UI Design

The UI design for this project is focused on a clean, modern, black-and-white feel. We wanted the platform to look different from other patient portals like MyChart, which can sometimes feel crowded or overwhelming. The main goal of the design is to make the system easier for both patients and providers to use while still including important healthcare features like appointments, messages, health records, lab results, prescriptions, and AI support.

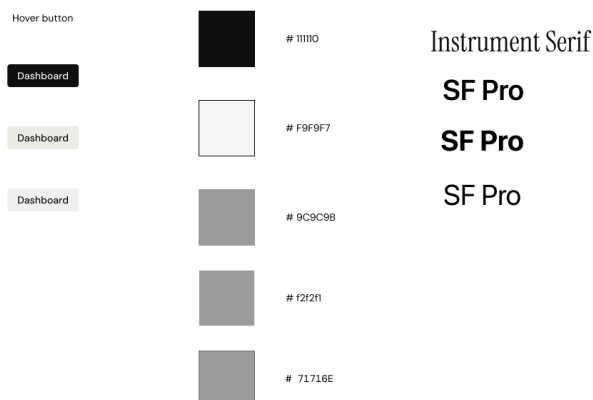
The design has two main sides: the provider dashboard and the patient dashboard. The provider dashboard is made for doctors and clinical staff, so it focuses on information they need to see quickly, such as the daily schedule, patient alerts, refill requests, lab summaries, pending encounters, and the AI assistant. The patient dashboard is focused more on helping patients manage their own care, including viewing appointments, checking medications, reviewing medical history, sending messages, and using the AI health assistant.

For the visual style, we used mostly black, white, and gray tones to keep the interface simple and professional. The typography combines Instrument Serif for large headings and SF Pro for readable body text and buttons. This creates a modern but still clinical appearance. The UI is organized around card-based sections, navigation tabs, and clear action buttons, so the information is separated into sections instead of being shown all at once.

The AI assistant is shown as a floating chat panel on both the patient and provider pages. On the provider side, it can help summarize patient information and answer questions about appointments, lab results, or patient history. On the patient side, it can help explain things like appointment preparation, medication instructions, lab results, and next steps in a more understandable way.

The appointment flow includes screens for selecting visit type, choosing a provider, selecting a date and time, reviewing appointment details, and confirming the appointment.

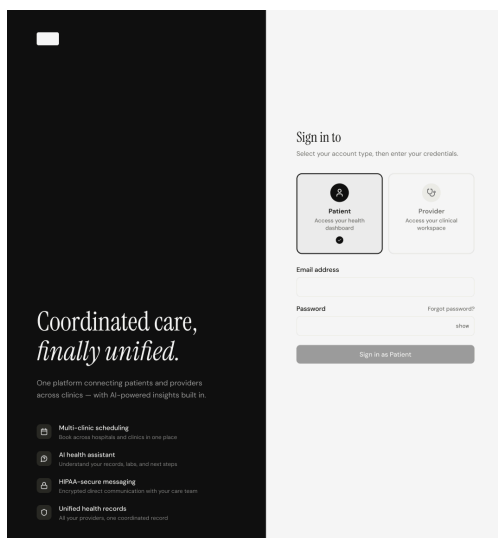
This design supports the purpose of the project by bringing patient care, provider tools, scheduling, messaging, health records, and AI assistance into one connected platform.



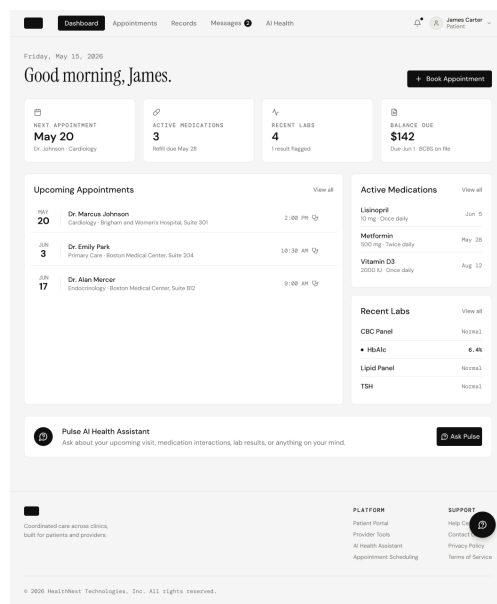
Color and Typography

Figma UI mockups link:

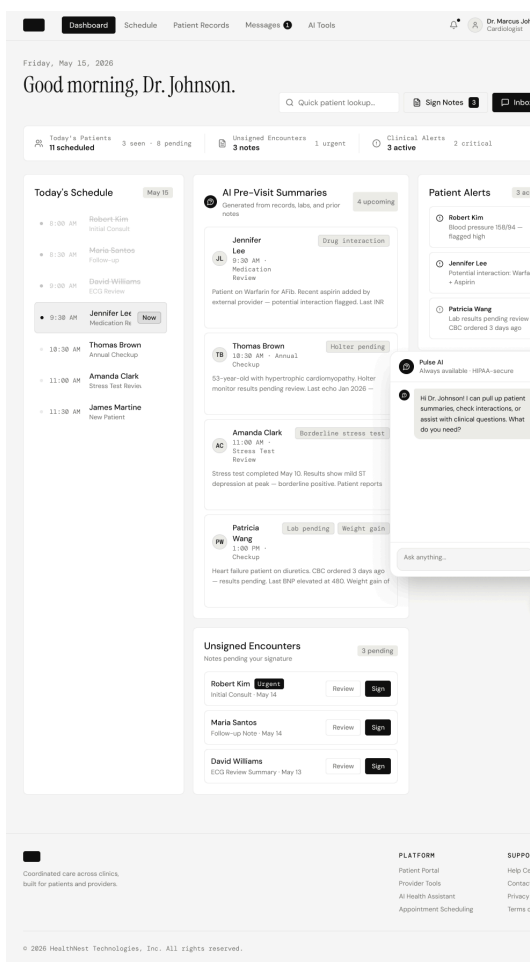
<https://www.figma.com/design/seHKWddNKI3xj3UgiyAXdl/UI?node-id=29-2&t=SaTsr6WhgntfBfqV-1>



Log in as patient



Patient Dashboard



AI open in doctor dashboard

5. Database Design

Supabase is the Postgres provider Healthnest is based upon. The following schema has been exported from Supabase to describe our high-level entities: Patient and Provider. These tables are automatically populated as users are onboarded onto Healthnest, through use of Postgres triggers, policies, and edge functions.

Table: patient_provider_relationships

Columns

Name	Type	Constraints
id	uuid	Primary
patient_id	uuid	
provider_id	uuid	
relationship	care_relationship	
started_at	timestampz	
ended_at	timestampz	Nullable
created_by	uuid	Nullable
created_at	timestampz	

Table: patients

Columns

Name	Type	Constraints
id	uuid	Primary
user_id	uuid	Unique
mrn	text	Unique
first_name	text	
last_name	text	
preferred_name	text	Nullable
date_of_birth	date	
sex_at_birth	sex_at_birth	
gender_identity	text	Nullable
phone_enc	bytea	Nullable
email_enc	bytea	Nullable

Name	Type	Constraints
address_line1_enc	bytea	Nullable
address_line2_enc	bytea	Nullable
city	text	Nullable
state	text	Nullable
postal_code_enc	bytea	Nullable
country	text	
emergency_contact_enc	bytea	Nullable
created_at	timestampz	
updated_at	timestampz	
deleted_at	timestampz	Nullable

Table: providers

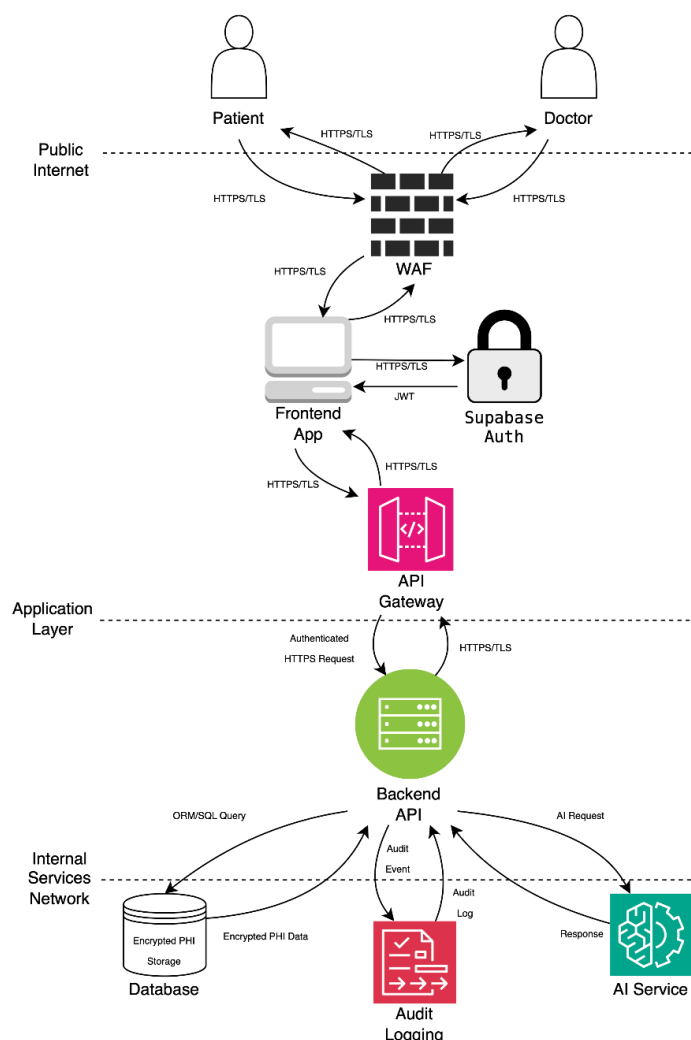
Columns

Name	Type	Constraints
id	uuid	Primary
user_id	uuid	Unique
npi	text	Nullable Unique
first_name	text	
last_name	text	
title	text	Nullable
specialty	text	Nullable
license_no_enc	bytea	Nullable
status	text	
created_at	timestampz	

Name	Type	Constraints
updated_at	timestampz	
deleted_at	timestampz	Nullable

6. Security Design

This section describes the security design for our application. For illustration purposes, a data flow diagram (DFD) is shown to the right. As you can see, the application will contain three primary trust boundaries, each one including its own set of security requirements and controls. As data passes between these trust boundaries, security mechanisms will be applied to ensure the confidentiality and integrity of sensitive personal health information (PHI). Throughout the system, HTTPS/TLS will be used to protect data in transit, while AES-256 encryption will be used to protect data at rest.



Authentication

There will be numerous security controls dedicated to secure authentication in our application. To login, a user will be required to use Multi-Factor Authentication (MFA). While time constraints may limit the number of MFA methods fully implemented in the prototype, a production-ready system could support multiple authentication factors, including username and password, one-time passcodes (OTP), biometric authentication, trusted device authentication, and external identity providers.

Once a user is authenticated, the server will issue a JSON Web Token (JWT) that the client will include with subsequent requests to authenticate and authorize access to

protected resources. The JWTs will be stored in cookies configured with the `HttpOnly`, `Secure`, and `SameSite` flags. These protections help ensure tokens are transmitted only over HTTPS/TLS connections, remain inaccessible to JavaScript, and improve session security by reducing exposure to cross-site scripting (XSS) and Cross-Site Request Forgery (CSRF) attacks. Additionally, Cross-Origin Resource Sharing (CORS) policies will be configured to restrict unauthorized cross-origin requests.

JWTs will have short expiration periods to reduce the risk associated with compromised access tokens. To maintain a seamless user experience, refresh tokens with longer expiration periods will also be issued to allow clients to obtain new access tokens without requiring users to repeatedly authenticate. Additionally, the system may implement server-side token revocation mechanisms to invalidate tokens before expiration in situations such as user logout or suspected account compromise.

Beyond technical authentication controls, the system will also enforce strong password policies to reduce the risk of credential-based attacks and unauthorized access. These policies may include minimum password length requirements, mandatory character complexity rules, password reuse restrictions, and periodic password reset requirements.

Authorization

Our application will implement Role-Based Access Control (RBAC) for authenticated users. The system will also follow the principle of least privilege to ensure users can access only the patient records and protected resources relevant to their assigned roles and responsibilities. These security controls are also essential for preserving the confidentiality, integrity, and availability principles of the CIA triad.

To enforce authorization policies, server-side authorization checks will be enforced on all protected API endpoints to prevent unauthorized access and privilege escalation attacks. Access to sensitive resources and patient records will be logged to support auditing, monitoring and HIPAA compliance requirements.

Input Validation and Sanitization

The application will follow a zero trust approach toward user input. All input will undergo multiple layers of validation and sanitization. Input will first be validated on the client side to reduce unnecessary or potentially malicious network requests, and then validated again on the server side before any requests are processed by the database. Additionally, all user-supplied content will be sanitized and treated as non-executable to help mitigate XSS attacks. Database queries will also use parameterized queries, either through an Object-Relational Mapping (ORM) framework or manual parameterization, to prevent SQL injection attacks.

Rate Limiting

The system will implement rate limiting at both the application and infrastructure levels. Application-level rate limiting will enforce business logic restrictions, including maximum login attempt policies, to reduce exposure to brute-force attacks and unauthorized authentication attempts. These protections, combined with strong password policies and Multi-Factor Authentication (MFA), will significantly improve account security.

At the infrastructure level, rate limiting and traffic filtering mechanisms implemented through the API Gateway and Web Application Firewall (WAF) will help mitigate denial-of-service (DoS) and Distributed Denial-of-Service (DDoS) attacks by restricting excessive or malicious traffic from individual sources. Infrastructure monitoring and logging systems will also support the detection and analysis of abnormal traffic patterns and potential attacks.

Monitoring, Logging and Error Handling

The system will generate logs to support auditing, monitoring, and incident detection. These logs will include application-level events such as login attempts, resource access attempts, authorization failures, and application errors. The system will ensure that sensitive Personal Health Information (PHI) and Personally Identifiable Information (PII) are not exposed within logs or error messages.

Infrastructure and network monitoring systems will also collect logs related to network traffic patterns, API activity, and database interactions. These logs may be aggregated within a centralized monitoring and analysis platform such as Splunk to support security monitoring and threat detection. Additionally, automated monitoring and alerting mechanisms will help identify suspicious traffic patterns and support mitigation of potential attacks or unauthorized activity. These monitoring systems will also support incident response and forensic analysis in the event of a potential security breach.

Network

The system infrastructure will implement multiple layers of network and infrastructure security controls to protect internal services and sensitive patient data. Web traffic entering the application will pass through a Web Application Firewall (WAF) and API Gateway to help filter malicious requests, enforce rate limiting policies, and centralize request validation.

Internal services and databases will remain isolated behind protected trust boundaries to reduce unauthorized access to sensitive resources. All communication between system components will occur over secure HTTPS/TLS connections to protect data transmitted across the network.

Data Protection

In addition to protecting data in transit through secure network protocols, the application will also protect data at rest. Sensitive data stored within databases and other storage systems will be encrypted using the industry-standard AES-256 encryption algorithm to help prevent unauthorized access and disclosure of sensitive information.

Compliance/Security Standards

The application will be designed with HIPAA compliance considerations in mind and will implement security controls intended to help mitigate the OWASP Top 10 Web Application Security Risks. Security best practices will be integrated throughout the software development lifecycle to ensure security remains a primary consideration during system design, development, testing, and deployment.

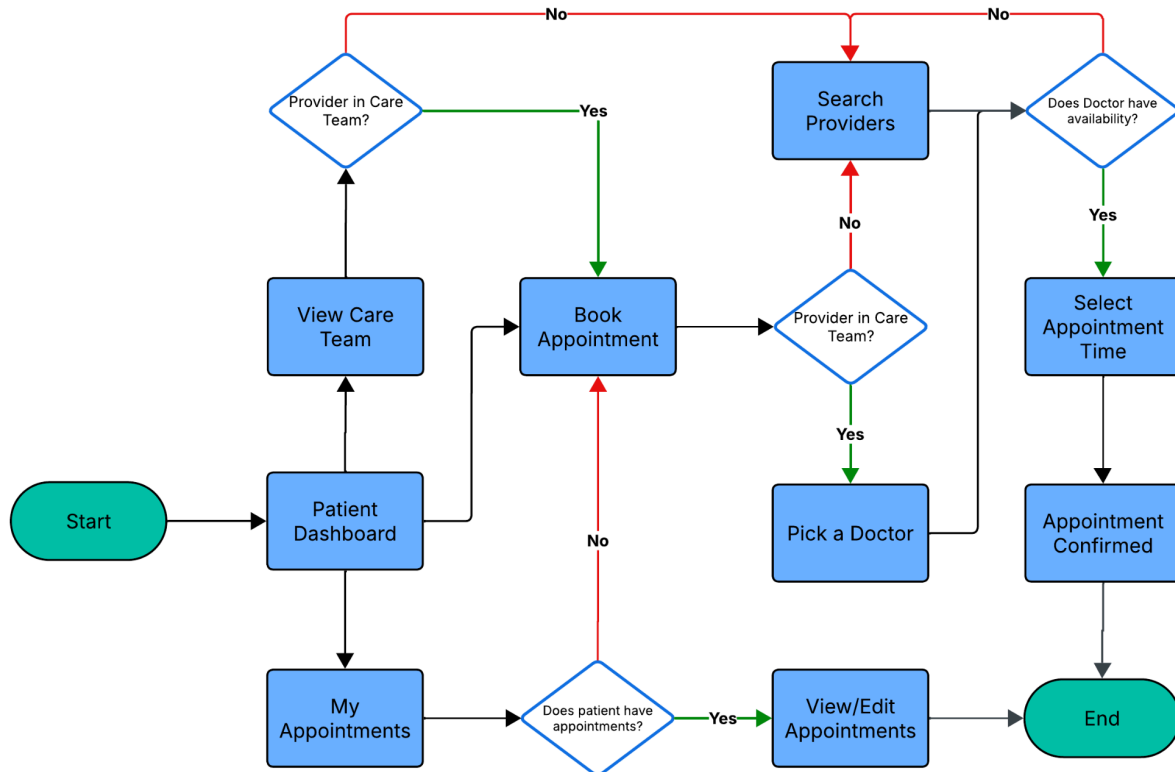
7. Business Logic and/or Key Algorithms

Login/Signup

The login and signup system uses two main authentication methods. Standard email/password authentication is handled through Supabase which hashes passwords for protections. When a successful login has occurred, Supabase issues an access token which the backend validates.

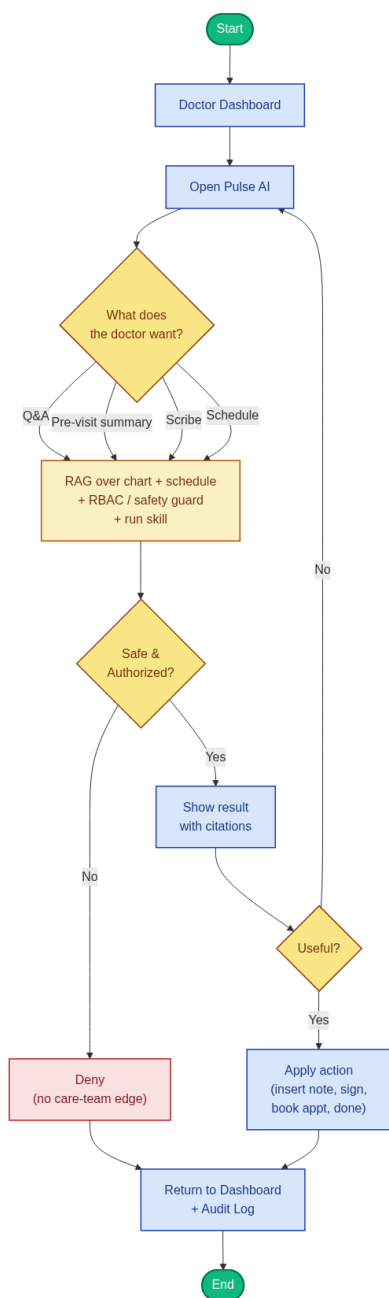
In the future we will implement WebAuthn for passkey/biometric authentication. During registration, the server generates a random challenge which the browser passes to the device authenticator which can be Face ID or fingerprint to create a key pair; one public key which is stored in the database and a private one which is stored on the user device. During login, the authenticator signs it with the newly generated challenge with the stored private key and both the front and backend communicate with each other using conversion functions to translate binary data used by the device's authenticator into regular text and back again.

Patient Facing Appointment Scheduler

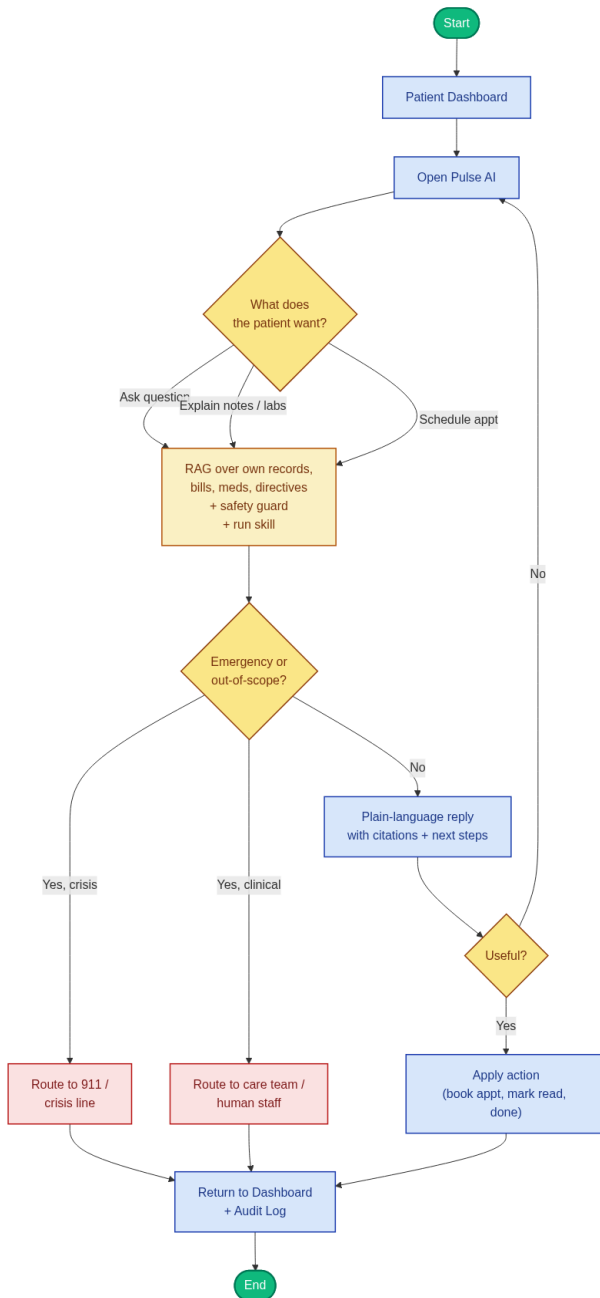


Pulse AI

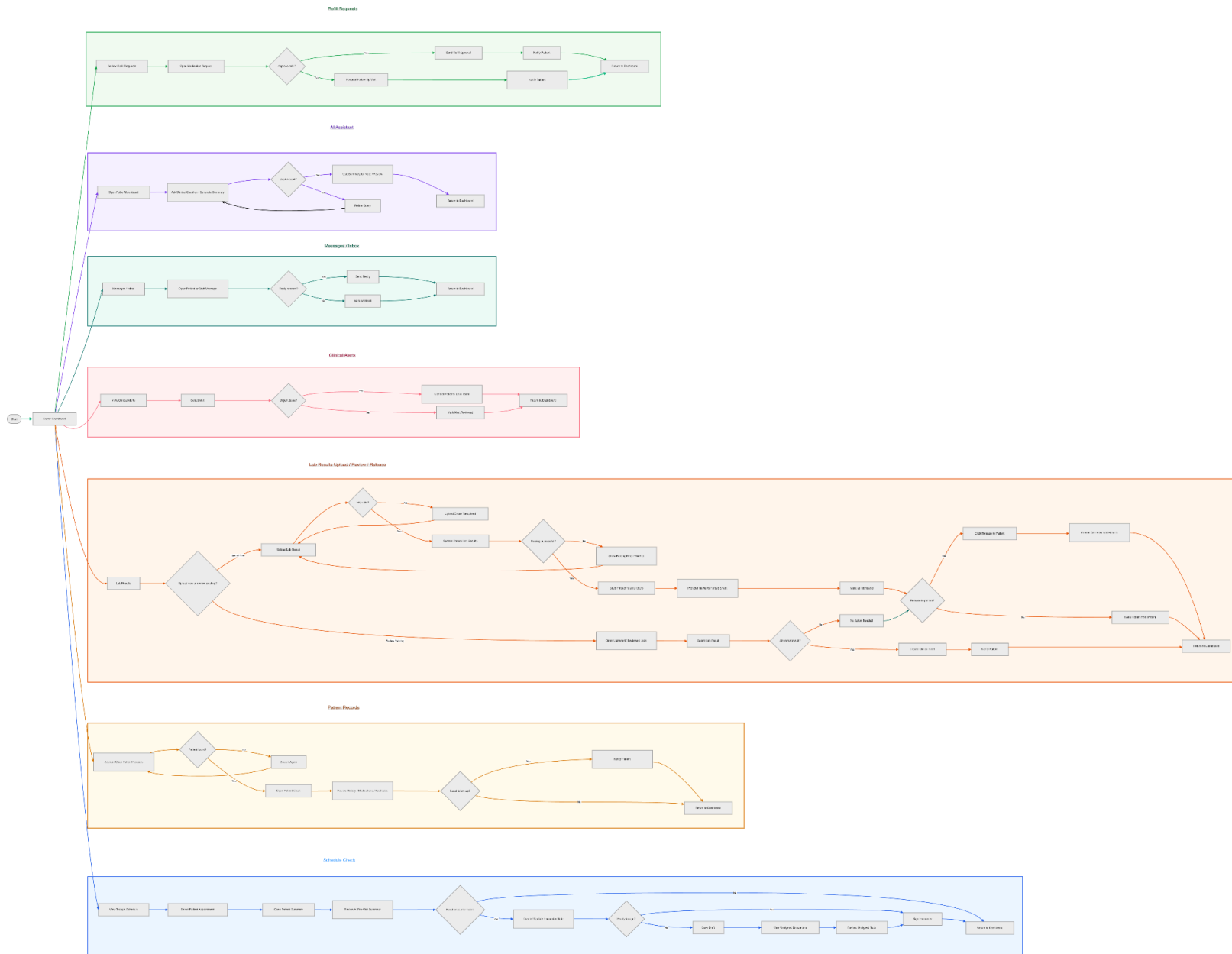
Pulse AI: Doctor-Facing Assistant (DFA)



Pulse AI: Patient-Facing Assistant (PFA)



Doctor Facing Side



8. Design Patterns

Our software has multiple design patterns for both our frontend and backend. Starting with the backend, we use a layered architecture where each file serves as different responsibilities. The `router.py` file handles HTTP routes and request wiring, while `service.py` contains the main business logic, validation, and response formatting. The `main.py` file handles application startup and environment-based configuration. We also use an Adapter/Mapper pattern in the service layer. Helper functions like `_serialize_user` and `_serialize_session` convert Supabase response data into the format expected by our API. We use a Singleton pattern for the Supabase client in `client.py` and data transfer objects in `schemas.py`.

On the frontend, we use component composition in React. Files like App.jsx, Login.jsx, PatientDashboard.jsx, and DoctorDashboard.jsx are separated into focused components, which makes the UI easier to reuse, test, and update. Our frontend also uses a Facade pattern in authApi.js. This file hides the details of authentication, such as fetch calls and localStorage, behind simple functions like signIn, signUp, signOut, and fetchUser. This gives the rest of the frontend a clean and stable interface for authentication. authApi.js also uses an Observer pattern with its listener system and onAuthStateChange function which allows multiple components to subscribe to authentication state changes and update automatically when the user logs in or logs out.

9. Rest APIs

Healthnest's REST API operates on a FastAPI framework using Python. It exposes the endpoints from the backend and consumes one external REST API (Supabase) for authentication, database, and file storage. All internal endpoints require a bearer token in the header unless explicitly noted.

Authentication API

The Authentication API wraps Supabase Auth and issues sessions used by the rest of the system.

Method	Endpoint	Description
POST	/auth/signup	Registers a new user and provisions either a patient or provider profile
POST	/auth/signin	Issues an access/refresh token pair.
POST	/auth/signout	Revokes the current session.
GET	/auth/me	Returns the user behind the supplied access token.
POST	/auth/refresh	Exchanges a refresh token for a fresh session.

Lab Results API

Role-aware endpoints to handle lab results. Providers can upload, edit, release, and archive. Patients can list and view only their own released results.

Method	Endpoint	Description
POST	/lab-results	Uploads a lab file. Auto-detects format if not provided.

Method	Endpoint	Description
PATCH	/lab-results/{lab_results_id}	Partial update of header fields and entries.
POST	/lab-results/{lab_results_id}/release	Releases a lab result to the patient.
POST	/lab-results/{lab_results_id}/archive	Archives a lab result.
GET	/lab-results	For providers, lists their patients lab results. For patients, lists their own released lab results.

Patients API

Endpoints used for patient lookup and typeahead in the UI.

Method	Endpoint	Description
POST	/patients	Returns patients within the provider's panel.
PATCH	/patients/{patient_id}	Returns a single patient by ID.

10. RAG Architecture

Pulse AI answers questions about a patient's own care, so its responses must be grounded in the patient's actual records rather than the model's general knowledge. HealthNest implements this with a Retrieval-Augmented Generation (RAG) subsystem inside the backend **ai/retrievalpackage**. The design splits cleanly into a write path (ingestion) that converts domain records into embedded text chunks, and a read path (retrieval) that finds the chunks most relevant to a user's question and injects them into the model's context. Embeddings are generated with OpenAI's **text-embedding-3-small** model (1,536 dimensions) and stored in Supabase Postgres using the pgvector extension, so the RAG store lives in the same database.

Ingestion is built around pluggable source adapters. Every adapter implements a shared **SourceAdapter** interface with one method, **load_documents(patient_id)**, that projects domain rows into flat-text **Document** objects carrying a stable **source_ref** such as **appointments:<uuid>**. A source catalog registers four sources, medical records, billing, insurance, and care-team directives, matching the retrieval scope defined in the requirements. The records adapter emits one document per appointment and one per released lab result, so that a retrieval citation always points at a specific record rather than a blob of patient history. The pipeline (**ingest_for_patient**) runs per patient and proceeds in four steps. First, each ready

adapter loads its documents. Second, a SHA-256 hash of each document's content is compared against the stored **content_hash**; unchanged documents are skipped entirely, and changed documents have their old chunks deleted before re-embedding, so repeated ingestion never duplicates data. Third, changed content is chunked with a paragraph-first splitter targeting ~1,200 characters with 150 characters of overlap (small enough that citations stay tight, large enough that a single retrieved chunk is useful on its own). Finally, chunks are embedded in batches of 100 per API request and inserted into rag_chunks.

Retrieval is the second stage of the shared assistant pipeline (guard → retrieve → tool loop → finalize) described in the AI Assistant component. After the safety guard passes, retrieved chunks are injected into the prompt as a dedicated system message with each chunk labeled by its **chunk_id** and source, instructing the model to cite the specific records it relied on. Both the patient-facing and doctor-facing assistants use the identical orchestrator. For the DFA, the patient context is the provider-selected patient passed in by the route, so a clinician's retrieval is scoped to one patient per conversation just as a patient's is.

11. AI usage Log

Section	Who	AI contribution (%)	Description	Verified by
6) Security Design	Aaron	40%	I used ChatGPT to help tighten up my wording for the security design portion of the document, also using it to add detail in some places.	Aaron
3	Deven	30%	Used Opus 4.7 to breakdown and try to find the holes in suggested architectures. It was able to bridge some gaps, like adding a top-level Dashboard Controller for simplicity and scalability.	Deven
4 UI Design	Stephanie	20%	I used ChatGPT to refine the wording, fixing grammar, and make sure all the key details were covered.	Stephanie
1 Introduction	Stephanie	10%	I used ChatGPT to refine the wording and grammar	Stephanie

Section	Who	AI contribution (%)	Description	Verified by
REST API	Deven	10%	Simply had Claude Code regurgitate the endpoints in the codebase in a clean manner. Once generated, I manually cross-checked.	Deven
2 - Software Architecture	Aaron	70%	I had Claude Code generate an architecture diagram for our application. I then used ChatGPT to assist in the description of the architecture. I read through all of the generated material and verified it manually.	Aaron